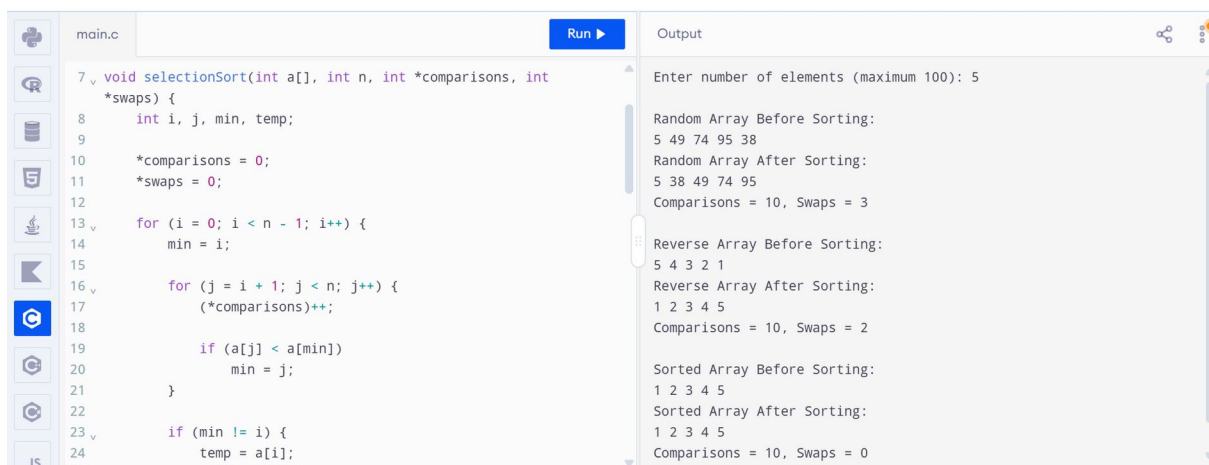


Aim

To implement Selection Sort on random, reverse-sorted, and already-sorted arrays and compare the number of comparisons and swaps performed by the algorithm.

Algorithm

1. Read the number of elements in the array.
2. Generate three arrays: random, reverse-sorted, and already-sorted.
3. For each array, find the minimum element in the unsorted portion.
4. Swap the minimum element with the first element of the unsorted portion.
5. Repeat until the entire array is sorted.
6. Count the comparisons and swaps for each array.
7. Display the sorted arrays and performance statistics.



The screenshot shows a C program in a code editor with a sidebar on the left containing icons for file explorer, search, and other IDE features. The code is in a file named 'main.c' and implements the Selection Sort algorithm. It includes a 'Run' button in the top right of the editor. The output window on the right shows the results of running the program for three different input arrays: random, reverse-sorted, and already-sorted. The program prompts the user to enter the number of elements (maximum 100), which is 5. For the random array, the output shows the array before and after sorting, with 10 comparisons and 3 swaps. For the reverse array, the output shows the array before and after sorting, with 10 comparisons and 2 swaps. For the sorted array, the output shows the array before and after sorting, with 10 comparisons and 0 swaps.

```
7 void selectionSort(int a[], int n, int *comparisons, int *swaps) {
8     int i, j, min, temp;
9
10    *comparisons = 0;
11    *swaps = 0;
12
13    for (i = 0; i < n - 1; i++) {
14        min = i;
15
16        for (j = i + 1; j < n; j++) {
17            (*comparisons)++;
18
19            if (a[j] < a[min])
20                min = j;
21        }
22
23        if (min != i) {
24            temp = a[i];
```

Output

Enter number of elements (maximum 100): 5

Random Array Before Sorting:
5 49 74 95 38
Random Array After Sorting:
5 38 49 74 95
Comparisons = 10, Swaps = 3

Reverse Array Before Sorting:
5 4 3 2 1
Reverse Array After Sorting:
1 2 3 4 5
Comparisons = 10, Swaps = 2

Sorted Array Before Sorting:
1 2 3 4 5
Sorted Array After Sorting:
1 2 3 4 5
Comparisons = 10, Swaps = 0

Complexity Analysis

- Best-case time complexity: $O(n^2)$
- Average-case time complexity: $O(n^2)$
- Worst-case time complexity: $O(n^2)$
- Auxiliary space complexity: $O(1)$ for the sorting algorithm.

Result: Selection Sort successfully sorts random, reverse-sorted, and already-sorted arrays. It performs $n(n-1)/2$ comparisons regardless of the initial order, but the number of swaps may vary.